PAL Agent: Setup Guide (v1)

PAL stands for **Prompt Abstraction Layer**.

Written for a beginner builder. Every decision below is a default you can change. The goal of v1 is a working loop: you submit inputs, PAL produces a brief, a plan, and build instructions, and you can give feedback that updates the result.

Runtime diagram: see the architecture diagram built from the `runtime` doc (four rows: intake, reasoning, execute, deliver, with the feedback loop back to intent).

---

1. What PAL does, in plain English

PAL takes what you give it (files, a prompt, links) and turns it into a clear plan and a set of instructions that another agent can follow. It works in stages, and each stage saves a file so you can see what it thought.

Your spec has two halves:

- **Intake and pipeline (the PAL core):** Compile inputs → Extract intent → Develop hypothesis → Enhance documentation → Generate instructions → Execute → Save → Present → Interpret feedback (loop).
- **The agent it builds (the "Agent Structure" half):** a sales/website/email agent with tools (send email, create website, search web, read/write files) and a response loop (read → extract intent → construct → test → execute → save → present → accept feedback).

Build the PAL core first. The Agent Structure is what PAL *outputs*, so it comes later.

---

2. Default decisions (change any of these)

Piece	Default	Why
Database	Supabase (Postgres)	Already in your locked stack. Stores prompts, links, and run history.
File storage	Supabase Storage bucket	Holds uploaded files. Keeps them private to you.
Brain	Claude API, model `claude-sonnet-5-5`	Handles every reasoning stage. Check the exact model string in docs.claude.com before coding. Your tech-stack note still lists an older model, so update it.
App shell	Next.js + Tailwind on Vercel	Your stack. One intake page plus one results page.
Email delivery	Resend	Already connected in your tools. Used in the Deliver step.
Orchestration	Plain code (no n8n in v1)	Fewer moving parts. Add n8n later if you want visual flows.

---

3. Build order (each phase ends with a check)

Phase 0: Accounts and keys (about 30 minutes)

- Create or confirm accounts: Supabase, Anthropic (API), Vercel, Resend.
- Put keys in a file called `.env.local` inside your project. Never paste keys into a chat or commit them to GitHub.
- Check: you can see each service's dashboard.

Phase 1: Database and storage

- In Supabase, run the SQL in Section 4.
- Create a private storage bucket named `pal-inputs`.
- Turn on Row Level Security (included in the SQL) so only your user can read your rows.
- Check: you can insert one test row from the Supabase table editor.

Phase 2: Intake (Compile Inputs)

- Build one form with: file upload, prompt text, links, file description, and format.
- On submit: upload files to `pal-inputs`, write a row to `pal_files`, write the prompt and links to `pal_prompts`.

- Generate `master.md`: a plain text file listing every input, its link, and its description. Save it to storage.
- Check: submit a test prompt and see `master.md` appear in the bucket.

Phase 3: Core reasoning stages Build each stage as one function that takes text in and returns text or JSON out. Save each output to `pal_runs` and to storage.

1. `extractIntent(master)` returns JSON: `{ wants, goal, deliverables, constraints, missing_info }`. `wants` is one of: `build_agent`, `update_website`, `migrate_backend`, `email_campaign`, `content_production`. This becomes `intent-summary.md`.
2. `developHypothesis(intent)` uses inductive reasoning and returns `output-hypothesis.md`: what the finished result probably looks like.
3. `enhanceDocs(intent, hypothesis)` uses deductive reasoning and web search to gather best practices. Output sections: by industry, by function, by segment, by use case, and best platforms by output. Each claim gets a source link.
4. `generateInstructions(all of the above)` returns `build-instructions.md`: system prompt, guardrails, capabilities, and prompt scripts for the final agent.

- Check: the four files exist and you can read them. Have a human (you) read them before moving on.

Phase 4: Execute, assess, and present

- Execute: run the build instructions against the Claude API with tools you define (write files, read files, search web, send email).
- Assess quality: a second, separate Claude call scores the result against the output hypothesis and lists gaps. Store the score in `pal_runs.summary`.
- Present: show the result on a results page and send it by Resend email.
- Check: one full run from intake to email.

Phase 5: Feedback loop

- Add a feedback box under each result.
- On feedback: extract the new intent, merge it into `master.md`, and rerun from the stage it affects. Save a new run row each time (never overwrite).
- Check: give feedback once and see a version 2 result.

---

4. Starter SQL (paste into Supabase SQL editor)

```sql
```

```sql
create table pal_sessions (
  id uuid primary key default gen_random_uuid(),
  user_id text not null,
  project_id text,
  created_at timestamptz default now()
);

create table pal_prompts (
  id uuid primary key default gen_random_uuid(),
  session_id uuid references pal_sessions(id) on delete cascade,
  user_id text not null,
  project_id text,
  question text,
  response text,
  input_links jsonb default '[]'::jsonb,
  file_storage_links jsonb default '[]'::jsonb,
  created_at timestamptz default now()
);

create table pal_files (
  id uuid primary key default gen_random_uuid(),
  session_id uuid references pal_sessions(id) on delete cascade,
  user_id text not null,
  project_id text,
  file_name text not null,
  file_format text,
  file_description text,
  storage_path text not null,
  metadata jsonb default '{}'::jsonb,
  created_at timestamptz default now()
);

create table pal_runs (
  id uuid primary key default gen_random_uuid(),
  session_id uuid references pal_sessions(id) on delete cascade,
  run_number int default 1,
  stage text not null,            -- intent | hypothesis | docs | instructions | e
  artifact_path text,
  summary jsonb default '{}'::jsonb,
  created_at timestamptz default now()
);
```

```
alter table pal_sessions enable row level security;
alter table pal_prompts  enable row level security;
alter table pal_files    enable row level security;
alter table pal_runs     enable row level security;
```

Note: you still need RLS policies that match your login before the app can read rows. Ask for those when you reach Phase 2. Without them, the app will show empty results.

---

5. Starter prompt for the intent stage

Paste this into the `extractIntent` call as the system prompt:

```
You are PAL's intent extractor. You will receive master.md, a list of raw user
inputs.
Return ONLY valid JSON with these keys:
- wants: one of build_agent | update_website | migrate_backend | email_campaign
| content_production | other
- goal: one sentence describing the outcome
- deliverables: list of concrete outputs
- constraints: list of limits (budget, tools, deadlines, brand rules)
- missing_info: list of questions that must be answered before building
Do not invent facts. If an input is unclear, put it in missing_info.
```

Use the same pattern for the other stages: one job per prompt, a stated output format, and an instruction not to invent facts.

---

6. Problems in the spec to fix before building

- The spec has a trailing line ("analyze it for value... draft a one-sheeter") that looks like a message to test, not a requirement. Treat it as a test case, not part of the build.
- The "Tool Calling:" section at the end of the Agent Structure is empty. Decide the tool list (Section 1 has a starting list) before writing the Execute stage.
- "Save response (index, timestamp summary json with ranking features - compressed)" is vague. For v1, save a plain JSON object with: run number, stage, timestamp, and a score. Add ranking later.
- Your tech-stack note defines PAL as "Parse, Abstract, Layer." That is out of date. PAL is Prompt Abstraction Layer.

---

7. Beginner safety rules

- Keys live in `.env.local` only. If one leaks, rotate it in the provider's dashboard.
- Test with a small input first (one short file, one sentence prompt). Only then try a full run.

- Do not connect PAL's email tool to a real contact list until the Execute stage has been tested with your own email address.
- Keep a copy of every working version. Git is the easiest way; ask for a step-by-step if you have not used it.

---

8. Open questions for you

1. Do you want PAL to live inside a Next.js app on Vercel, or as an n8n workflow first?
2. Should the Execute stage be allowed to send email automatically, or should it always wait for your approval?